Predicting TTC Subway Delays Using Machine Learning

Sherouk Rashed (20596339) - Hayam Rezk (20596917) **Group 3**

1 Abstract

TTC subway delays affect hundreds of thousands of Toronto commuters every day. This project builds a machine learning pipeline to predict whether a recorded subway incident will turn into a delay using only the information available at the time of incident logging: station, line, time of day, direction, incident code, and vehicle ID. We engineered 57 features from the 2025 TTC Subway Delay dataset and compared seven classifiers which are Logistic Regression as a baseline, Gradient Boosting, XGBoost, LightGBM, Random Forest, Extra Trees, and KNN. Six out of seven models exceeded our recall threshold of 0.80, with LightGBM reaching the best F1 at 0.7495 at its optimal threshold. The single strongest predictor turned out to be the incident cause code frequency: how often a given cause code appears in the training data.

2 Introduction

The TTC is the backbone of transit in Canada's largest city, carrying out 420 million trips in 2024, roughly 1.15 million rides daily [1]. A single delayed train can ripple across the network in minutes, which makes early prediction operationally useful since dispatchers who know a delay is likely can act before it cascades.

The few studies that have used the TTC dataset directly have focused on exploratory analysis delay patterns by station and time of day [2] rather than building predictive models.

More recent studies have shown that gradient-boosted trees outperform linear baselines on structured transit data by picking up nonlinear interactions between station load, time of day, and incident type [3], and that rolling lag features encoding a station's recent delay history tend to carry the strongest predictive signal [4].

We built on this by running a systematic comparison of seven classifiers, optimizing each model's decision threshold using the precision-recall curve, and comparing feature importance across all models to see which signals hold up consistently. Our starting hypothesis was simple: delays are not random. They cluster around specific stations, time windows, and incident types, and a model trained on those patterns should hit at least 0.8 recall on held-out data. We chose recall as the hard constraint because a missed delay forces dispatchers into reactive mode, while a false alarm costs only a brief check.

2.1 Objective

This project develops a machine learning pipeline to predict whether a TTC subway incident will cause a delay using only the metadata available at the moment of logging. We compare 7 classifiers against a recall target of 0.8, treating missed delays as more costly than false alarms, and identify the features that drive delay risk consistently across all models.

3 Method

3.1 Dataset

We used the 2025 TTC Subway Delay data from the City of Toronto Open Data Portal [6]. The stations' names in the raw data were a mix of real station names, street addresses. We cleaned them against a whitelist of 74 known

TTC stations and dropped anything that didn't match. We also removed a handful of non-subway line entries and obvious typos. After pre-processing the final dataset was reduced to 25,095 incidents down from 25,713, a 2.4% drop that barely moved the class balance (35.4% to 36.3% delay rate).

The binary target `DELAY_OCCURRED` is 1 when the recorded minimum delay is greater than zero. Delays occurred in about 36% of incidents, giving a roughly 1.83:1 class imbalance.

3.2 Feature Engineering

10 raw columns are expanded into 57 features across seven groups:

- **Temporal**: hour of day with cyclical sine/cosine encoding so midnight and 11:00 PM are treated as adjacent hours, rush-hour flags for the 7:00 AM - 9:00 AM and 4:00 PM - 7:00 PM windows, week days, and weekend indicators.
- **Station**: historical delay rate per station, total incident count, and a flag for the top-10 busiest stations.
- **Line and Direction**: per-line delay rates, one-hot flags for YU, BD, and SHP lines, and directional flags (N/S/E/W/bidirectional).
- **Vehicle**: a flag for whether a specific train was logged at all (vehicle ID 0 means platform or infrastructure incident), the first digit of the vehicle ID as a rough fleet-series proxy, and a per-vehicle historical delay rate.
- **Incident Code**: a cause category derived from the two-character prefix of each TTC code (MU = mechanical, SU = signal, TU = track, PU = passenger, EU = electrical), a code frequency score, and binary flags for the 10 most common codes. Across all six models, `code_frequency` ranked as the single strongest predictor with an average normalised importance of 0.927.
- **Rolling Lag**: the 50-incident rolling delay rate per station, the 100-incident rolling rate per line, and time since the last incident at the same station and line. These features capture the current operational state rather than historical averages.
- **Interaction Terms**: rush hour $\times$ weekday (highest-risk operating window) and late night $\times$ weekend (lowest-risk window).

All rolling features are computed in chronological order to avoid data leakage, each row only sees incidents that happened before it.

3.3 Preprocessing

The data were split chronologically 80/20, with the first 80% used for training and the last 20% held out for testing. This mirrors real deployment: the model is always predicting incidents it has never seen, in a time period it was not trained on. Numeric features were scaled using RobustScaler with the 5th - 95th percentile range, fitted on training data only. We chose RobustScaler because delay durations have a long right tail that distorts standard z-score scaling. Class imbalance was handled through class weighting: `class_weight='balanced'` for most models, a `sample_weight` array for Gradient Boosting, and `scale_pos_weight` for XGBoost.

3.4 Models

Logistic Regression serves as an explicit baseline. Because it is restricted to linear decision boundaries, any meaningful gain from a nonlinear model reflects genuine structure in the data rather than just better hyperparameter tuning. The remaining six models span three families: sequential boosting (Gradient Boosting, XGBoost [3], LightGBM [5]), parallel bagging (Random Forest [7], Extra Trees), and a geometric approach (KNN). Each model is first run with sensible defaults, then tuned via RandomizedSearchCV with 30-50 iterations, 3-fold TimeSeriesSplit, and F1 as the scoring metric.

4 Experiments

We ran four experiments. First, we compared all models before and after tuning at the default 0.50 threshold, using Logistic Regression as the reference point.

Second, we swept the precision-recall curve to find the threshold that maximizes F1 for each model, producing an optimized-threshold leaderboard alongside the default one.

Third, we examined the confusion matrices with attention to False Negatives as the primary failure mode.

Fourth, we extracted feature importances from all six non-KNN models, normalized them to [0,1], and averaged across models to find features that are consistently predictive regardless of which algorithm is used.

5 Results

Table 1 shows key metrics at the default 0.50 threshold after tuning.

Table 1: Performance at default threshold (0.50). * = baseline.

Model	Recall	Precision	F1	PR-AUC	Specificity
Logistic Regression*	0.8491	0.5630	0.6771	0.6438	0.5997
Gradient Boosting	0.8651	0.6579	0.7474	0.8030	0.7268
XGBoost	0.8221	0.6743	0.7409	0.8053	0.7589
LightGBM	0.8535	0.6681	0.7495	0.8045	0.7425
Random Forest	0.8018	0.6933	0.7436	0.8063	0.7846
Extra Trees	0.8744	0.6506	0.7461	0.7977	0.7147
KNN	0.5435	0.5685	0.5557	0.5958	0.7495

Six out of seven models clear the 0.80 recall target at the default threshold, confirming our hypothesis. KNN is the only model that fails, recording a recall of 0.5435. LightGBM leads with the best F1 at 0.7495, while Logistic Regression ranks last among the real models at 0.6771. The gap between LightGBM and Logistic Regression is 0.072 F1 points, a difference unlikely to be explained by hyperparameter tuning alone. It reflects non-linear structure in the data that a linear model simply cannot capture regardless of how it is configured.

KNN finishes last on every metric, which tells us that Euclidean distance is not a good proxy for delay risk in this feature space. After threshold optimization, all seven optimal thresholds fall below 0.50, meaning the models already favour recall and lowering the threshold further adds negligible F1 gain. The largest improvement across all models is just +0.0015 for Gradient Boosting. KNN degenerates completely at its optimal threshold of 0.000, predicting every incident as delayed and achieving recall of 1.0 with precision of only 0.38 — a result that confirms its unsuitability rather than demonstrating any real discriminative ability.

Across all six models with importance measures, `code_frequency` comes out on top by a clear margin, with an average normalised importance of 0.927. The incident cause code — not the station or time of day — is the strongest and most consistent predictor of whether a delay will follow. `code_encoded` ranks second at 0.554, followed by specific code flags such as `code_MUIS` at 0.433. Rolling features such as `station_rolling_delay_rate` and `line_rolling_delay_rate` remain informative but rank lower than the incident code group across all models.

6 Conclusion

TTC subway delays are predictable from incident metadata alone. Six out of seven models exceed 0.80 recall at the default threshold on the held-out test set, confirming that delays are not random events but follow learnable patterns tied to incident cause, station history, and time of day. For production deployment we would recommend LightGBM since it achieved the best overall balance across all metrics, leading in both F1 at 0.7495 and PR-AUC

at 0.8045, while offering significantly faster inference than Gradient Boosting or XGBoost at equivalent depth. The deployment setup should maintain a live `code_frequency` score and station rolling delay rate, retrain monthly on the most recent 12 months of data, and recalibrate the decision threshold quarterly as operating patterns shift.

6.1 Limitations

- **Single year of data:** The dataset covers only 2025, meaning the model has seen each seasonal pattern exactly once.
- **Records lost during cleaning:** Station name cleaning removed 6.3% of records.
- **Occurrence only, not duration.** The model predicts whether a delay happens, not how long it would be. A 2-minute delay and a 45-minute delay both count as positives, yet they carry entirely different consequences and likely have different underlying causes.

6.2 Future work

- **Delay propagation.** The model currently predicts each station in isolation. In reality, a delay at one station cascades to neighboring ones, and modeling that chain would be considerably more useful operationally.
- **External feeds.** Plugging in weather APIs, TTC maintenance schedules, and event calendars as additional features would address one of the most significant gaps in the current feature set.
- **Multi-year data.** Training on several years of data would expose the model to a wider range of seasonal patterns and improve confidence in features that depend on historical trends.

References

[1] Toronto Transit Commission (2025). 34 billion rides and counting - the TTC reaches new service milestone. https://www.ttc.ca/news/2025/April/34-billion-rides-and-counting

[2] Schleifer, A. (2022). Toronto subway delays vary based on specific ridership patterns. *Telling Stories With Data*. https://tellingstorieswithdata.com/inputs/pdfs/paper_one-2022-alyssa_schleifer.pdf

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proc. 22nd ACM SIGKDD*, 785–794. https://doi.org/10.1145/2939672.2939785

[4] Ding, C., Wang, D., Ma, X., & Li, H. (2016). Predicting short-term subway ridership and prioritizing its influential factors using gradient boosting decision trees. *Sustainability*, 8(11), 1100. https://doi.org/10.3390/su8111100

[5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154. https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf

[6] City of Toronto Open Data Portal. (2025). TTC Subway Delay Data. https://open.toronto.ca/dataset/ttc-subway-delay-data/

[7] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324